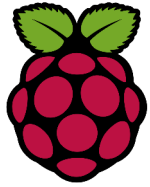


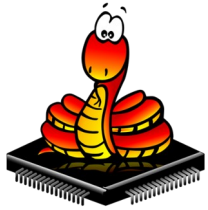


Atelier Raspi

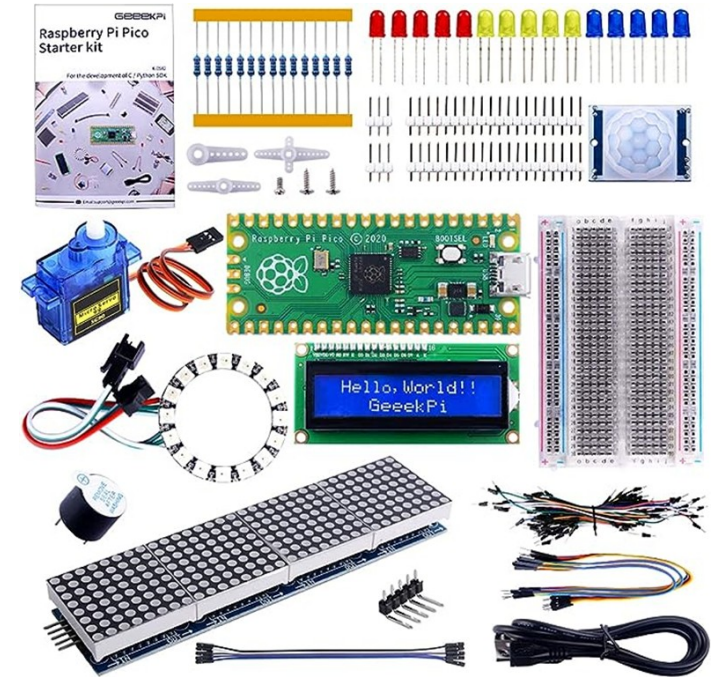
Atelier N°18 La mini serre instrumentée Boucle de Régulation



Logo du Raspberry Pico



Logo du MicroPython

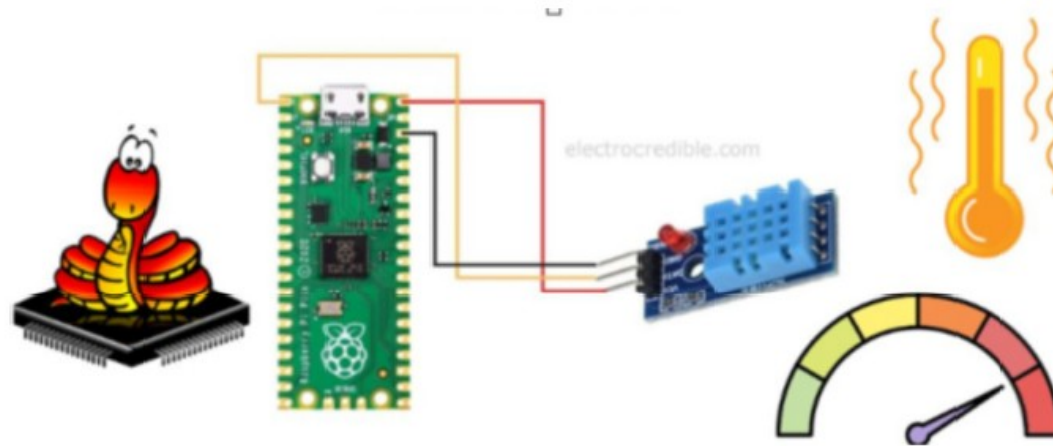


L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

Atelier N°18 La mini serre instrumentée

Boucle de Régulation

- Etape 1 Rappel des éléments du projet
- Etape 2 Rappel de la séance précédente
- Etape 3 Les boucles de régulation
- Etape 4 La régulation d'humidité
- Etape 5 Le programme (1/2)
- Etape 6 Le programme (2/2)



A18E1 Rappel des éléments du projet

Le cerveau de la serre

Carte MicroPython → le “cerveau” du système 🤖

Elle exécute le programme et prend les décisions.

Le module interface homme-machine

Il permet de communiquer avec la serre :

Écran LCD → affiche les informations

4 boutons → envoyer des commandes

4 LEDs → indiquer des états

Ruban de 8 LEDs → afficher des niveaux ou des animations 🌈

Les capteurs : DHT11 ou DHT22

Ces capteurs mesurent la température 🌡️ et l'humidité 💧

La carte MicroPython lit ces informations pour savoir comment réagir.

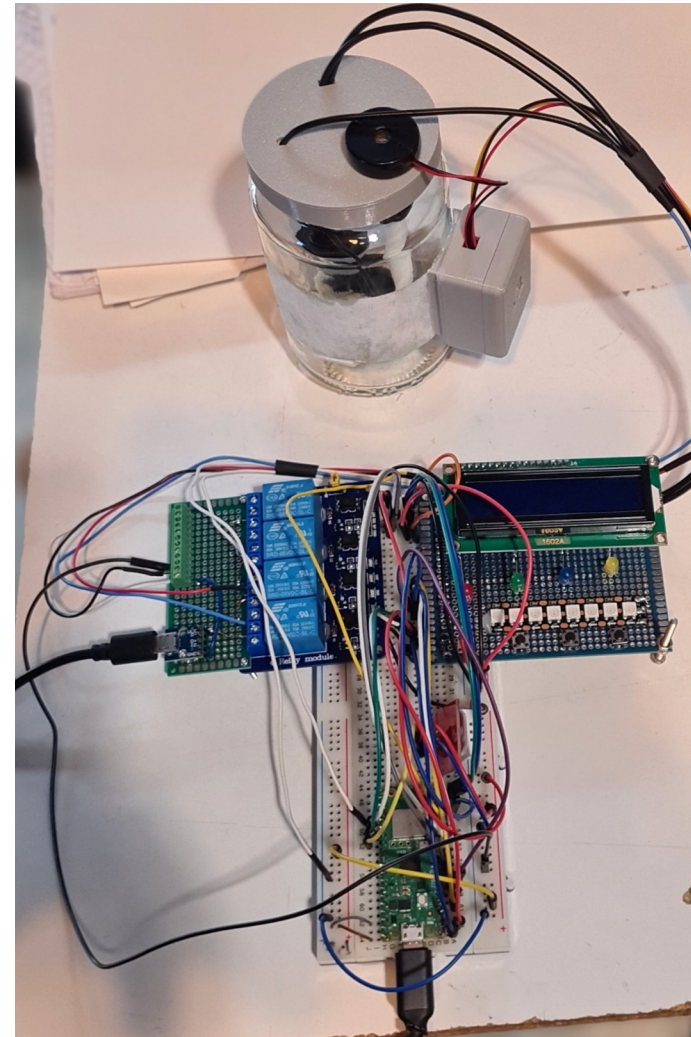
Les actionneurs : Carte 4 relais

Les relais permettent d'allumer ou d'éteindre des appareils électriques.

Relais du brumisateurs → ajoute de l'humidité 💧

Relais de la pompe → fait circuler l'eau 🌊

Relais du ventilateur → enlève de l'humidité



*Nous n'utiliserons pas le servo moteur qui ouvre une trappe
et le détecteur de présence vu pour le sapin de Noël.*

A18E2 Rappel de la séance précédente

Transition avec la séance précédente

Lors de la séance précédente, nous avons utilisé les boutons BP1 et BP2 pour tester les relais et commander la pompe manuellement.

BP1 → allumer la pompe

BP2 → arrêter la pompe

Le programme fonctionnait comme une petite machine à états :

état « éteint »

état « allumé »

👉 C'était l'utilisateur qui décidait quand changer d'état en appuyant sur les boutons.

Aujourd'hui, on change de fonctionnement

Cette fois, les boutons ne vont plus commander directement les relais.

C'est le programme qui prendra les décisions automatiquement grâce aux mesures du capteur d'humidité.

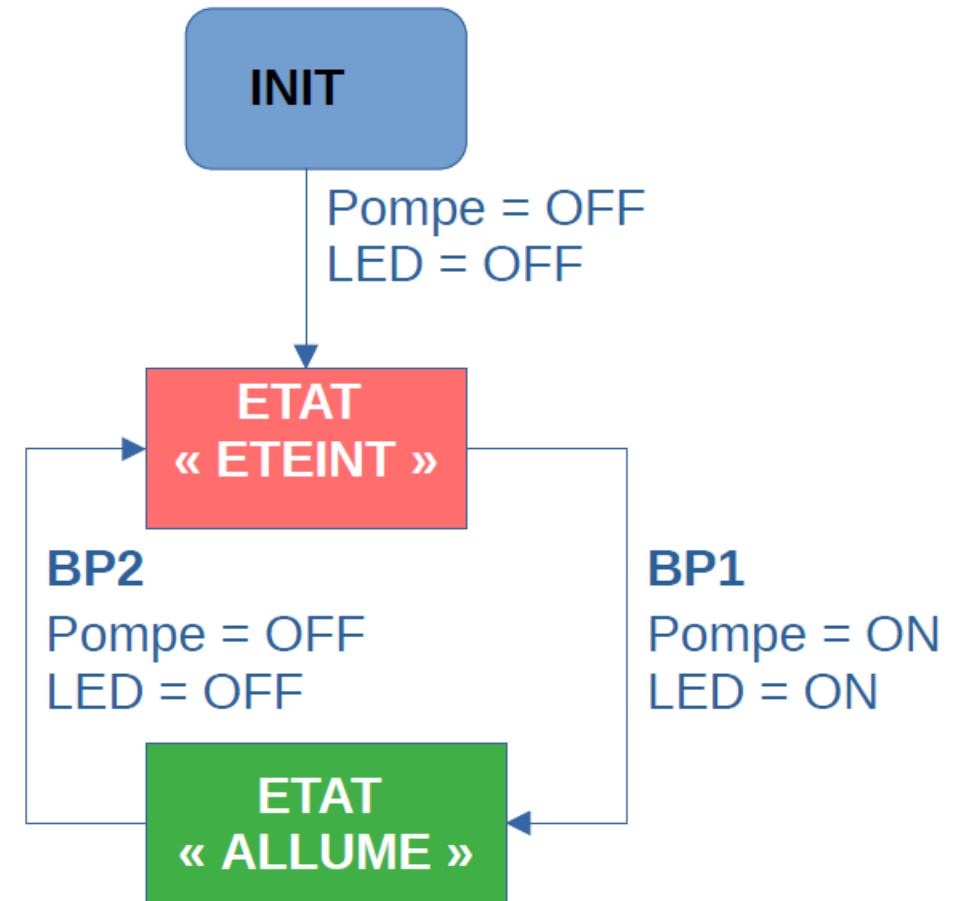
Au lieu de :

« J'appuie sur un bouton pour allumer »

le programme fera :

« L'air est trop sec, donc j'allume le brumisateurs »

Test du relais / pompe avec BP1 et BP2



A18E3 Les boucles de régulation

On passe à une boucle de régulation 💧

Le programme va maintenant :

- mesurer l'humidité,
- comparer avec des seuils,
- décider quoi faire,
- commander automatiquement les relais.

La serre devient alors autonome 🌱🤖

On peut créer plusieurs régulations :

Régulation de température 🌡️

Si la température est trop haute → on active le ventilateur

Si la température est trop basse → on pourrait activer un chauffage

Régulation d'humidité 💧

Si l'air est trop sec → on active le brumisateur

Si l'air est trop humide → on active le ventilateur

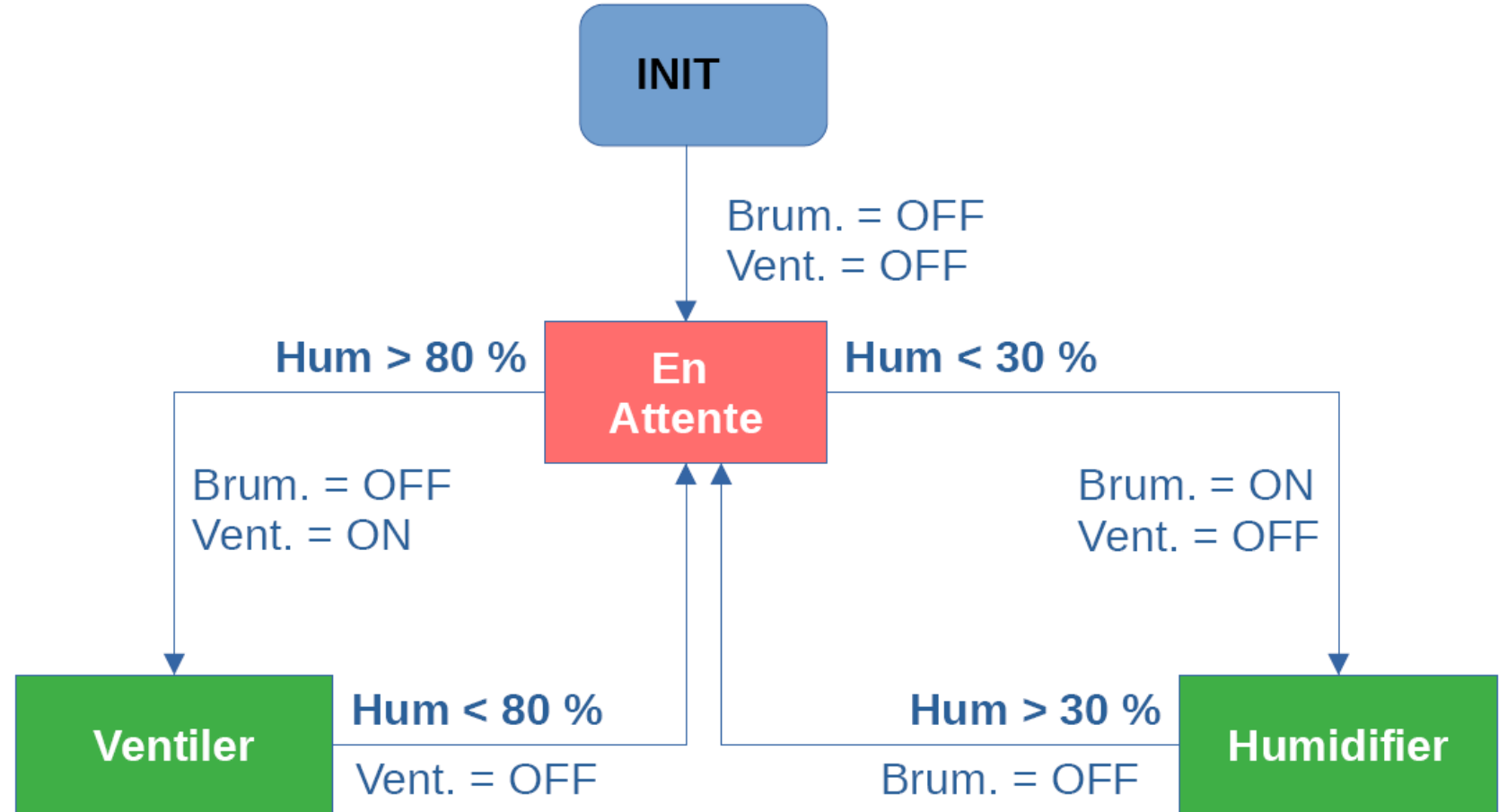
A18E4 La régulation d'humidité

Si l'air est trop sec → on active le brumisateur

Si l'air est trop humide → on active le ventilateur

Notre programme va :

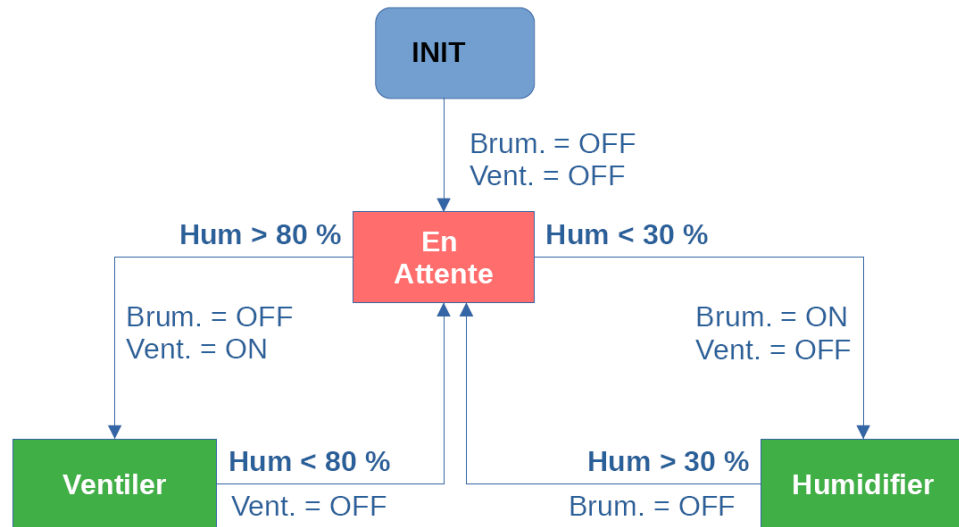
- mesurer l'humidité de l'air,
- décider quoi faire,
- piloter automatiquement les relais. 🤖



A18E5 Le programme

```
1 from machine import Pin
2 import time
3 import dht
4
5 time.sleep(0.1) # Wait for USB to become ready
6
7 # CONSTANTES
8 # definit les seuils d'humidite bas et haut
9 SEUIL_HUMIDITE_BAS=58
10 SEUIL_HUMIDITE_HAUT=60
11
12 # FONCTIONS
13 def afficher_humidite( valeur ):
14     print(f"humidite : {valeur:.1f}%")
15
16 def allumer_brumisateur( relais ):
17     relais.value(0)
18     time.sleep(0.1)
19     relais.value(1)
20
21 def eteindre_brumisateur( relais ):
22     relais.value(0)
23     time.sleep(0.1)
24     relais.value(1)
25     time.sleep(0.5)
26     relais.value(0)
27     time.sleep(0.1)
28     relais.value(1)
29
30 def allumer_ventilateur( relais ):
31     relais.value(0)
32
33 def eteindre_ventilateur( relais ):
34     relais.value(1)
35
```

A18E6 Le programme



```
37 # PROGRAMME PRINCIPAL
38 print("Boucle regulation humidite !")
39
40 # INIT
41 relais4_ventilateur = Pin(9, Pin.OUT)
42 relais2_brumisateur = Pin(7, Pin.OUT)
43 #DHT = dht.DHT11(Pin(22))
44 DHT = dht.DHT22(Pin(22))
45 time.sleep(1) # attente que le capteur humidite soit operationnel
46
47 # place les actionneurs dans leur etat par default
48 eteindre_ventilateur(relais4_ventilateur) # eteint le ventilateur
49
50 mon_etat = "attente" # variable d'etat du programme : "humidifie" / "attente" / "ventille"
51
52 # boucle principale
53 while True:
54     # on prend la mesure
55     DHT.measure()
56
57     # on affiche le niveau d'humidite
58     afficher_humidite( DHT.humidity() )
59     #print(f"etat {mon_etat}")
60
61     # en fonction de notre etat
62     if mon_etat == "attente":
63         # si l'humidite est en dessous du seuil bas
64         if DHT.humidity() < SEUIL_HUMIDITE_BAS:
65             # on allume le brumisateur
66             allumer_brumisateur(relais2_brumisateur)
67             # on indique notre nouvel etat
68             mon_etat = "humidifie"
69         elif DHT.humidity() > SEUIL_HUMIDITE_HAUT:
70             # on allume le ventilateur
71             allumer_ventilateur(relais4_ventilateur)
72             # on indique notre nouvel etat
73             mon_etat = "ventille"
74
75     elif mon_etat == "humidifie":
76         # on regarde si on est passe au dessus du seuil d'humidite pour arreter le brumisateur
77         if DHT.humidity() > SEUIL_HUMIDITE_BAS:
78             # on eteint le brumisateur
79             eteindre_brumisateur(relais2_brumisateur)
80             # on indique notre nouvel etat
81             mon_etat = "attente"
82
83     elif mon_etat == "ventille":
84         # on regarde si on est passe en dessous du seuil d'humidite pour arreter le ventilateur
85         if DHT.humidity() < SEUIL_HUMIDITE_HAUT:
86             # on eteint le ventilateur
87             eteindre_ventilateur(relais4_ventilateur)
88             # on indique notre nouvel etat
89             mon_etat = "attente"
```